

LAB 03 — Word Representations and Embeddings

Môn học: Xử lý ngôn ngữ tự nhiên và ứng dụng | Học kỳ: Học kỳ I - 2026

Giảng viên / TA: Phạm Ngọc Hải ▪ Hạn nộp: 23:59 - 07/10/2026

Chủ đề: From Sparse Representations to Dense Word Embeddings

1. Bối cảnh

Ở LAB 01, sinh viên biểu diễn văn bản bằng **TF-IDF**.

Ở LAB 02, sinh viên sử dụng frequency/count để xây dựng **language model**.

Hai lab đều dựa nhiều vào thống kê xuất hiện của từ. Tuy nhiên, các biểu diễn này có một hạn chế quan trọng:

Hai từ có nghĩa gần nhau không nhất thiết có vector gần nhau.

Ví dụ:

doctor
physician

có thể xuất hiện trong những ngữ cảnh tương tự, nhưng TF-IDF không tự động tạo ra quan hệ:

$$\text{doctor} \approx \text{physician}.$$

Word embedding đưa ra một cách tiếp cận khác:

Biểu diễn một từ bằng một vector dense sao cho những từ xuất hiện trong các ngữ cảnh tương tự có xu hướng có vector tương tự.

Lab này bắt đầu từ **distributional hypothesis**, xây dựng embedding đơn giản từ co-occurrence, sau đó chuyển sang **Word2Vec**, similarity, analogy và cuối cùng là phân tích giới hạn của static embeddings.

2. Mục tiêu học tập

Sau khi hoàn thành lab, sinh viên có thể:

- **LO1:** Giải thích distributional hypothesis.
- **LO2:** Xây dựng ma trận word-context từ corpus.
- **LO3:** Phân biệt sparse count representation và dense embedding.
- **LO4:** Tính cosine similarity giữa word vectors.
- **LO5:** Hiểu ý tưởng CBOW và Skip-gram.
- **LO6:** Sử dụng và phân tích Word2Vec.
- **LO7:** Đánh giá embedding bằng similarity và analogy.
- **LO8:** Phân tích ảnh hưởng của corpus, context window và vector dimension.
- **LO9:** Nhận diện hạn chế của static word embeddings.
- **LO10:** Giải thích tại sao contextual embeddings và Transformer trở thành bước tiếp theo.

3. Cấu trúc lab

Giai đoạn	Thời lượng	Hoạt động
Theory recap	20 phút	Distributional hypothesis

Giai đoạn	Thời lượng	Hoạt động
Calculation	25 phút	Co-occurrence và cosine
Prediction	10 phút	Dự đoán similarity
Experiment 1	25 phút	Word-context matrix
Core implementation	40 phút	Co-occurrence embedding
Experiment 2	30 phút	Word2Vec
Evaluation	25 phút	Similarity + analogy
Application	25 phút	Semantic search / clustering
Error analysis	20 phút	Embedding failures
Reflection	10 phút	Static → contextual embeddings

4. Theory recap

4.1 Distributional Hypothesis

Một ý tưởng nền tảng của NLP:

Words that occur in similar contexts tend to have similar meanings.

Ví dụ:

```
The doctor treated the patient.  
The physician treated the patient.
```

Hai từ:

```
doctor  
physician
```

xuất hiện trong những context tương tự.

Do đó có thể biểu diễn:

```
doctor    → vector  
physician → vector
```

sao cho:

$\text{similarity}(\text{doctor}, \text{physician})$

cao.

Đây là nền tảng trực tiếp của word embeddings.

5. Từ co-occurrence đến vector

Cho corpus:

```
the cat eats fish  
the cat likes milk  
the dog eats meat  
the dog likes fish
```

Chọn vocabulary:

```
cat
dog
eats
likes
fish
milk
meat
```

Với context window = 1, ta có thể xây dựng word-context matrix.

Ví dụ:

Word	cat	dog	eats	likes	fish	milk	meat
cat	0	0	1	1	0	0	0
dog	0	0	1	1	0	0	0
eats	1	1	0	0	1	0	1
likes	1	1	0	0	1	1	0

Sinh viên cần nhận ra:

 $\vec{cat}$

và

 $\vec{dog}$

có thể giống nhau vì chúng có context tương tự.

6. Bài tập tính toán

Bài 1 — Co-occurrence

Cho:

```
the cat eats fish
the dog eats fish
the cat likes milk
the dog likes meat
```

Context window:

$$k = 1.$$

Hãy xây dựng word-context vector cho:

```
cat
dog
eats
likes
```

Không sử dụng Python.

Bài 2 — Similarity

Cho:

$$x = [1, 2, 1]$$

và:

$$y = [2, 4, 2].$$

Tính:

$$\cos(x, y).$$

Sau đó giải thích:

Hai vector có độ lớn khác nhau nhưng cosine similarity bằng 1 có ý nghĩa gì?

7. Bài 3 — So sánh semantic similarity

Cho:

$$v_{doctor} = [0.8, 0.1, 0.7]$$

$$v_{physician} = [0.7, 0.2, 0.8]$$

$$v_{banana} = [-0.2, 0.9, -0.1].$$

Tính:

$$\cos(doctor, physician)$$

và:

$$\cos(doctor, banana).$$

Dự đoán từ nào gần doctor hơn trước khi tính.

8. Bài 4 — Sparse vs dense

Cho một vocabulary gồm:

$$V = 10,000.$$

Một word-context representation có:

$$10,000$$

chiều nhưng chỉ có 30 non-zero entries.

Một embedding có:

300

chiều và hầu hết các thành phần đều khác 0.

Sinh viên trả lời:

1. Biểu diễn nào sparse?
2. Biểu diễn nào dense?
3. Vì sao dense representation có thể thuận lợi hơn cho semantic similarity?
4. Dense representation có chắc chắn “tốt hơn” trong mọi bài toán không?

Câu cuối rất quan trọng: **không biến embedding thành một khẩu hiệu “dense = tốt hơn”.**

9. Prediction trước experiment

Sinh viên phải dự đoán trước khi chạy model.

Prediction 1

Những từ nào gần nhau nhất?

```
doctor  
physician  
hospital  
banana  
car
```

Prediction 2

Nếu context window tăng từ:

 $2 \rightarrow 5,$

similarity có thay đổi không?

Prediction 3

Nếu embedding dimension:

```
50 → 100 → 300
```

thì chất lượng có chắc chắn tăng không?

Prediction 4

Từ:

```
doctor  
physician
```

có chắc chắn gần nhau không nếu corpus chỉ có 100 câu?

Sinh viên phải ghi:

```
Prediction:  
Reason:  
Confidence:
```

10. Experiment 1 — Word-context representation

Sử dụng corpus nhỏ trước.

Sinh viên xây dựng:

$$X \in \mathbb{R}^{|V| \times |V|}$$

với:

```
rows    = target words  
columns = context words
```

Thử:

```
window = 1  
window = 2  
window = 5
```

So sánh:

- vocabulary size;
- matrix size;
- number of non-zero values;
- similarity giữa các từ.

11. Core implementation

Sinh viên tự implement:

```
build_vocabulary()  
build_cooccurrence_matrix()  
cosine_similarity()  
most_similar()
```

Ví dụ:

```
most_similar(  
    word="doctor",  
    matrix=X,  
    vocabulary=vocab,  
    top_k=5  
)
```

Không được sử dụng trực tiếp:

```
Word2Vec(...)
```

ở phần implementation cốt lõi.

Mục tiêu là sinh viên hiểu:

```
context
  ↓
co-occurrence
  ↓
vector
  ↓
similarity
```

12. Từ co-occurrence matrix đến embedding

Đặt câu hỏi:

Nếu vocabulary có 100,000 từ thì word-context matrix có vấn đề gì?

Ta có:

$$100,000 \times 100,000$$

entries.

Ngay cả khi rất sparse, representation vẫn có thể rất lớn.

Đây là động lực để tìm kiếm:

low-dimensional dense representation

thay vì giữ nguyên không gian co-occurrence khổng lồ.

13. Neural Word Embeddings

Giới thiệu ngắn:

```
Co-occurrence statistics
  ↓
Dense vector representation
  ↓
Neural language modeling
  ↓
Word2Vec
```

Không cần bắt sinh viên chứng minh toàn bộ thuật toán Word2Vec.

Nhưng phải hiểu ý tưởng.

14. CBOW

CBOW:

Context → target word

Ví dụ:

the ____ eats fish

Context:

the
eats
fish

Model dự đoán:

cat

Sơ đồ:

```
graph TD
    A[context words] --> B[embeddings]
    B --> C[aggregation]
    C --> D[prediction]
    D --> E[target word]
```

15. Skip-gram

Skip-gram đảo chiều:

Target word → **context words**

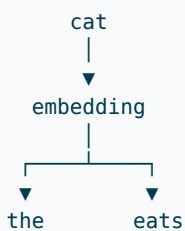
Ví dụ:

cat

dự đoán:

the
eats

Sơ đồ:



Sinh viên cần trả lời:

CBOW và Skip-gram khác nhau ở input/target như thế nào?

16. Bài tập prediction — CBOW vs Skip-gram

Cho câu:

the cat eats fish

với window = 1.

Hãy liệt kê training examples cho:

CBOW

Skip-gram

Không chạy code.

Mục tiêu là sinh viên phải tự tạo được training pairs.

17. Experiment 2 — Word2Vec

Sử dụng một implementation chuẩn như:

```
Word2Vec(...)
```

nhưng sinh viên phải ghi rõ:

- corpus;
- vocabulary;
- embedding dimension;
- window;
- min count;
- epochs;
- training objective.

Ví dụ:

```
vector_size = 100  
window = 5  
min_count = 2  
epochs = 10
```

Không được chỉ viết:

“Tôi dùng Word2Vec.”

18. Inspect embeddings

Với các từ:

```
doctor  
hospital  
patient  
disease  
computer  
football  
banana
```

tìm:

```
model.wv.most_similar("doctor")
```

và ghi lại Top-5.

Sinh viên phải giải thích:

Tại sao những từ này gần doctor?

Không chấp nhận câu trả lời:

“Vì model học được semantic similarity.”

Phải chỉ ra **evidence từ corpus/context**.

19. Experiment 3 — Context window

Train ba model:

```
window = 2  
window = 5  
window = 10
```

So sánh similarity.

Ví dụ:

Word pair	Window 2	Window 5	Window 10
doctor-physician			
doctor-hospital			
cat-dog			

Câu hỏi:

Window lớn hơn có luôn tốt hơn không?

20. Experiment 4 — Embedding dimension

So sánh:

```
50  
100  
300
```

dimensions.

Đánh giá:

- training time;
- model size;
- similarity;
- analogy;
- downstream task đơn giản.

Không cần yêu cầu benchmark quá lớn.

Mục tiêu là sinh viên hiểu trade-off:

dimension $\leftrightarrow$ capacity $\leftrightarrow$ data $\leftrightarrow$ computation.

21. Evaluation — Word similarity

Cho một danh sách word pairs:

```
doctor – physician  
car – automobile
```

```
king - queen  
cat - dog  
computer - banana
```

Model tạo similarity score.

Sinh viên phải:

1. xếp hạng;
2. so sánh với intuition;
3. tìm những trường hợp bất ngờ.

Không nên đánh giá embedding chỉ bằng:

“Nhìn kết quả thấy khá tốt.”

Phải có quantitative result.

22. Evaluation — Word analogy

Giới thiệu:

$$\textit{king} - \textit{man} + \textit{woman} \approx \textit{queen}.$$

Hoặc:

$$\textit{Paris} - \textit{France} + \textit{Italy} \approx \textit{Rome}.$$

Sinh viên thực hiện:

```
most_similar(  
    positive=["king", "woman"],  
    negative=["man"]  
)
```

Sau đó giải thích:

Tại sao vector arithmetic có thể biểu hiện một số quan hệ?

Quan trọng:

Không được yêu cầu sinh viên kết luận rằng embeddings “hiểu” quan hệ như con người.

Đây là pattern trong vector space, không phải bằng chứng về human-like understanding.

23. Bài tập tính analogy

Cho các vector giả định:

$$\textit{king} = [8, 2, 7]$$

$$\textit{man} = [5, 1, 5]$$

$$\textit{woman} = [5, 3, 5].$$

Tính:

$$\textit{king} - \textit{man} + \textit{woman}.$$

Sau đó giải thích vector mới có thể đại diện cho loại quan hệ nào.

24. Application — Semantic Search

Liên kết trực tiếp với LAB 01.

LAB 01:

```
Query
↓
TF-IDF
↓
Cosine
↓
Documents
```

LAB 03:

```
Query words
↓
Word embeddings
↓
Semantic similarity
↓
Related words/documents
```

Sinh viên xây dựng một semantic search đơn giản.

Ví dụ query:

```
medical treatment
```

và tìm các documents chứa các từ semantic gần:

```
therapy
clinical
patient
treatment
```

Đây là bước chuẩn bị rất tốt cho:

sentence/document embeddings → semantic retrieval → RAG.

25. Error analysis

Chọn:

- 3 similarity đúng;
- 3 similarity sai/bất ngờ.

Ví dụ:

```
doctor → nurse
doctor → hospital
```

doctor → disease

Sinh viên phân tích:

Observed:
Expected:
Possible explanation:
Evidence from corpus:

Các nguyên nhân có thể:

- corpus nhỏ;
- domain bias;
- frequency;
- polysemy;
- context window;
- noisy data;
- insufficient training;
- vocabulary limitation.

26. Một phần rất quan trọng — Polysemy

Cho từ:

bank

Trong:

I deposited money in the bank.

và:

We sat on the river bank.

Static Word2Vec chỉ có **một vector** cho:

bank

Đặt câu hỏi:

Làm thế nào một vector duy nhất có thể đại diện cho hai nghĩa?

Đây chính là một giới hạn quan trọng của static embeddings.

27. Reflection — Từ Word2Vec đến Transformer

Sinh viên phải hoàn thành bảng:

Representation	Context-dependent?	Sparse/Dense	Một từ có nhiều vector?
TF-IDF	...	...	...
Co-occurrence	...	...	...
Word2Vec	...	...	...
Contextual embedding	...	...	...

Sau đó trả lời:

Tại sao bank cần contextual representation?

28. AI policy

AI được phép:

- giải thích API;
- debugging;
- visualization;
- documentation;
- kiểm tra implementation;
- đề xuất cách tối ưu code.

Không được dùng AI cho:

- calculation ban đầu;
- prediction;
- interpretation của similarity;
- error analysis;
- reflection;
- individual learning check.

Nếu dùng AI:

AI assistance statement

Tool:

Purpose:

Generated content:

Modified content:

Verification:

29. Individual learning check

Mỗi sinh viên khoảng 3 phút.

Random một câu:

Distributional hypothesis là gì?

hoặc:

Tại sao doctor và physician có thể gần nhau?

hoặc:

CBOW khác Skip-gram ở đâu?

hoặc:

Tại sao tăng context window có thể vừa tốt vừa xấu?

hoặc:

Tại sao Word2Vec không phân biệt được hai nghĩa của bank?

hoặc:

Tại sao TF-IDF không phải word embedding?

30. Deliverables

```
lab03/
├── README.md
├── calculations.md
├── prediction.md
├── cooccurrence.py
├── word_embedding.ipynb
├── results.csv
├── error_analysis.md
└── reflection.md
```

31. Rubric

Thành phần	Điểm
Theory & calculation	10
Prediction	10
Co-occurrence implementation	15
Word2Vec experiment	15
Hyperparameter experiment	15
Evaluation	15
Application	10
Error analysis	5
Reflection	3
Individual learning check	2
Tổng	100

Ở lab này trọng số **implementation Word2Vec** giảm so với implementation co-occurrence.

Lý do là sinh viên **không cần tự viết lại toàn bộ Word2Vec optimization**. Cái các em cần hiểu là:

context → co-occurrence → vector → similarity

và sau đó:

CBOW / Skip-gram → learned dense representation

32. Mạch kiến thức của ba lab đầu

Đến đây ba lab đã tạo thành một chuỗi đẹp:

```
LAB 01
Text
↓
Count
↓
TF-IDF
↓
```

Sparse representation
↓
Lexical search

LAB 02
Text
↓
Count
↓
Conditional probability
↓
N-gram LM
↓
Perplexity
↓
Next-word prediction

LAB 03
Text
↓
Context
↓
Co-occurrence
↓
Dense embedding
↓
Semantic similarity
↓
Word2Vec

Và sau LAB 03, **LAB 04** rất tự nhiên nên chuyển sang **Text Classification**:

TF-IDF → Word Embedding → Document Representation → Classification.